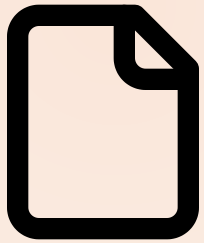
MODULE 08 · 24 MIN

Refactoring & Documentation at Scale

Claude Code Bootcamp · Day 1 · Block 8 of 10



Promise

In 24 minutes you will:

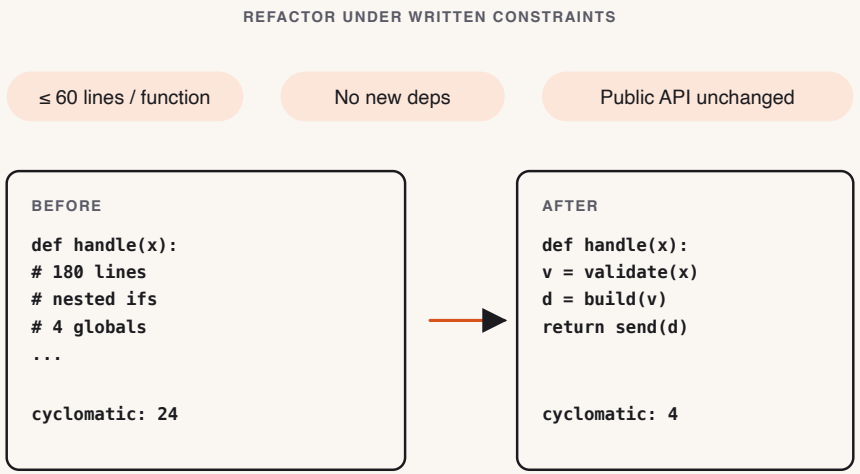
1. Refactor a deliberately messy Python module into something readable.
2. Stay inside hard **written constraints** so Claude doesn't run away with it.
3. Ship `HANDOFF.md` and `ARCHITECTURE.md` from the diff.

Why this matters

- Refactoring is where AI either delivers huge value or burns a day. The difference is **constraints**.
- Documentation written from a fresh diff is more accurate than documentation written months after the fact.
- Senior engineers are rated on what they *don't* change. Same here.

Concepts

- **Constraint-bounded refactor**: tell Claude exactly what may *not* change. Public API, file count, runtime behavior — pick what to lock.
- **Two-pass workflow**: first pass = refactor; second pass = generate docs from the diff. Never combined.
- `HANDOFF.md`: a one-pager for the next engineer — what was done, why, watch-outs.
- `ARCHITECTURE.md`: shape of the module — components, data flow, dependencies. One diagram (ASCII OK), not five.



Live demo flow

1. Instructor opens `exercises/part-08/before/` — a messy Python module on purpose.
2. First, the **bad refactor**: prompt without constraints. Show the bloated diff.
3. Reset. Now the **constrained refactor** prompt. Show the small, targeted diff.
4. Run tests — still green.
5. Second pass: generate `HANDOFF.md` from the diff. Read aloud.

Mini project

Refactor `exercises/part-08/before/` and ship `HANDOFF.md` + `ARCHITECTURE.md`.

Deliverables under `module-08/`:

- `after/` — refactored source
- `HANDOFF.md`
- `ARCHITECTURE.md`
- `constraints.md` — the exact constraint list you used

Step-by-step lab

STEP 1 Copy `exercises/part-08/before/` to `module-08/after/`.

STEP 2 Read the messy module. Decide what is allowed to change and what isn't. Write `constraints.md` first.

STEP 3 Run the **constrained refactor** prompt with `constraints.md` as input.

STEP 4 Apply Claude's diff. Re-run the existing tests (provided in `before/`).

STEP 5 Run the **HANDOFF** prompt against the diff.

STEP 6 Run the **ARCHITECTURE** prompt against the refactored code.

STEP 7 Edit both docs. Commit.

Suggested Claude Code prompts

```
CONSTRAINED REFACTOR
You will refactor the module below for readability only.

HARD CONSTRAINTS
- No new files. No new dependencies.
- Public function signatures unchanged. Module-level imports unchanged.
- Behavior on all existing tests must be byte-identical.
- Replace nested conditionals with early returns where it shortens code.
- Rename local variables only when the new name is materially clearer.
- No comments unless they explain a non-obvious *why*.

Output: a unified diff. No prose around it.
```

```
HANDOFF.md
Generate a one-page HANDOFF.md from the diff below. Sections:
- What changed (3 bullets max)
- Why
```

✓ Deliverable checklist

- [ ] `module-08/after/` passes the existing tests in `before/`.
- [ ] `module-08/constraints.md` was written *before* the refactor.
- [ ] `HANDOFF.md` ≤ 40 lines, has all four sections.
- [ ] `ARCHITECTURE.md` ≤ 80 lines, has a diagram and component paragraphs.

✓ Definition of done

✅ Tests still green · ✅ Diff respects every constraint · ✅ Both docs would orient a new engineer in 5 minutes.

Review checkpoint

Pair (60 s each):

STEP 1 Pick one constraint your partner wrote. Did the diff actually respect it?

STEP 2 Read partner's `HANDOFF.md`. Could you take over the module from it?

Common mistakes

- Skipping `constraints.md`. Without it Claude rewrites everything; you'll spend the lab reading.
- Combining refactor + docs in one prompt. The docs end up describing the prompt, not the diff.
- Letting Claude "improve while you're at it". Veto every unrequested change.
- 200-line `ARCHITECTURE.md`. Trim aggressively.

Instructor notes

- 5 / 5 / 12 / 2 split.
- Show the unconstrained refactor first; the lift is what sells the technique.
- If short, drop `ARCHITECTURE.md`; ship `HANDOFF.md` only.
- The "before" directory must keep existing tests; verify before delivery.

Transition to next module

We have shipped real artefacts in 7 modules. Now we capture the *patterns* — turn them into reusable Claude Skills you can carry to any project.

Next: Module 9 — Commands, Hooks & Reusable Workflows.